
Firmware

Secure Software & Hardware Systems

Overview

- Introduction to Firmware
 - Definition
 - Classification
- Inner workings
 - ARM ISA
 - Memory Maps
 - Programming Models
 - Interrupts
 - Hardware interactions
- Extraction
 - Firmware File Formats
 - Showcased Techniques
- Recap

Overview

- Introduction to Firmware
 - Definition
 - Classification
- Inner workings
 - ARM ISA
 - Memory Maps
 - Programming Models
 - Interrupts
 - Hardware interactions
- Extraction
 - Firmware File Formats
 - Showcased Techniques
- Recap

A general Note on Firmware

Software

Firmware

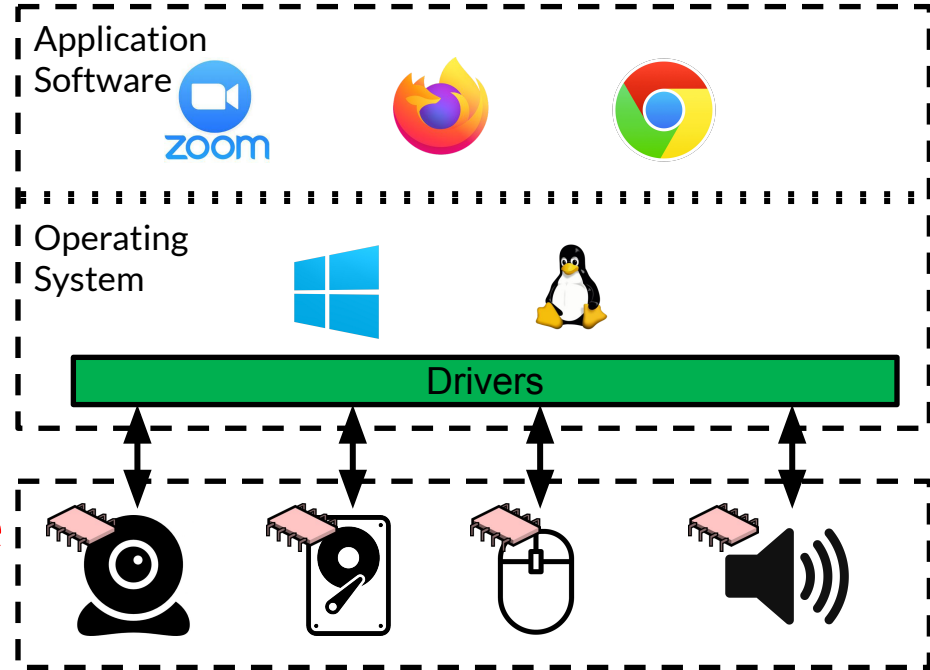
Hardware

Firmware (on your computer)

Software

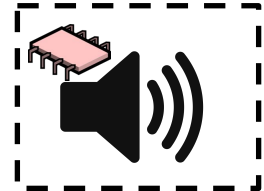
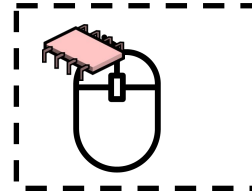
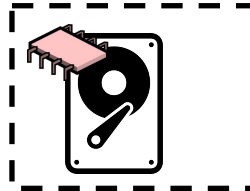
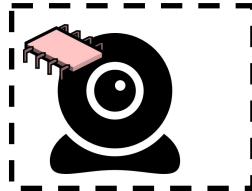
Hardware

Firmware



Firmware (on embedded systems)

- Low-level code driving the system
 - Application specific
 - Direct HW interactions
- Stored in non-volatile memory
 - E.g., EEPROM
 - E.g., flash memory



Types of embedded systems

- The variety of embedded systems is tremendous
 - What does, e.g., “deeply embedded” even mean?
- To discuss FW effectively, we need a classification!
 - ISA?
 - Size?
 - Application?
- Let's use an OS-centric classification
 - Good proxy for complexity and security measures



Types of Firmware

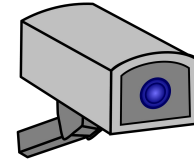
Type-I:

General purpose OS-based



Type-II:

Embedded OS-based



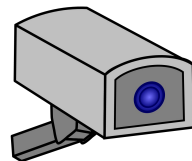
Type-III:

No OS-Abstraction



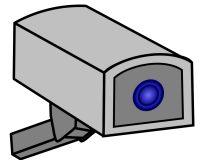
OS-centric classification: Advantages

- Tells us immediately about FW complexity
- Gives intuition about possible security features
- Can be applied to embedded system as well!



OS-centric classification: Disadvantages

- No insight about ISA and HW platform
- Same HW platform can be different types in different systems
- Type-II/Type-III can be hard to distinguish



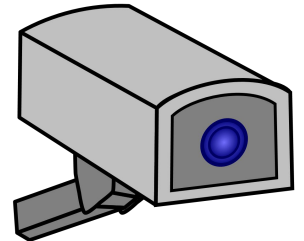
Type I – General purpose OS-based

- Popular OS retrofitted for embedded system
- Hardware interaction abstracted by kernel
- Example: Linux on a router
 - uclibc
 - busybox
 - No loadable kernel modules
 - Special drivers for the system's unique hardware
 - Custom system calls

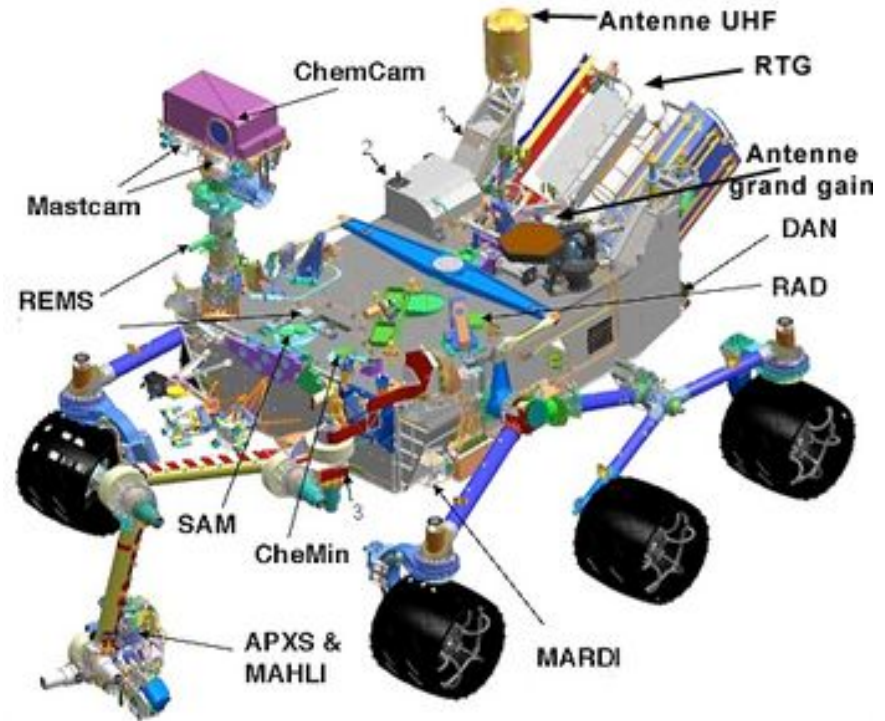


Type II – Embedded OS-based

- Custom OS for embedded systems
- Created to solve custom challenges of the system:
 - E.g., Real-Time constraints
- Reduced functionality compared to traditional OS:
 - Eg., No MMU/virtual memory
- Example: VxWorks on NASA's Curiosity Rover
 - 2 computers with 256kB EEPROM, 256MB DRAM, 2GB flash
 - Send to Mars in 2012
 - Still operational!



Type II – Embedded OS-based



Type III – No OS Abstraction

- Can be entirely custom
- May include “OS-library” with OS functions, but:
 - System & application code are compiled together
 - No strict kernel/user separation
- Example: Firmware on Logitech G600 Mouse*
 - HW based on Atmel ATmega32
 - >12k lines of disassembly
 - Code for USB, SPI, EEPROM and more



*J. Maskiewicz et al. “Mouse Trap: Exploiting Firmware Updates in USB Peripherals.” WOOT’14.

Types of Firmware

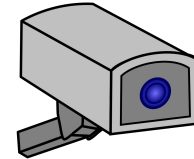
Type-I:

General purpose OS-based



Type-II:

Embedded OS-based



Type-III:

No OS-Abstraction



Types of Firmware

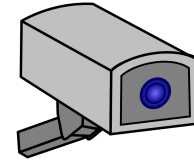
Type-I:

General purpose OS-based



Type-II:

Embedded OS-based



Type-III:

No OS-Abstraction



Overview

- Introduction to Firmware
 - Definition
 - Classification
- Inner workings
 - ARM ISA
 - Memory Maps
 - Programming Models
 - Interrupts
 - Hardware interactions
- Extraction
 - Firmware File Formats
 - Showcased Techniques
- Recap

The ARM Instruction Set Family

- Advanced RISC Machine (ARM)
 - Developed by Arm Holdings
 - Licensed to other companies
 - First product in 1985
- Today:
 - Market leader for mobile devices (phones, tablets)
 - Most popular architecture for 32-bit embedded systems*
 - Becoming present on the desktop market
- Multiple versions of ISA, e.g.:
 - ARMv9
 - ARMv8
 - ARMv6-m



* Embedded Survey. "The Current State of Embedded Development." May 2023

ARM Profiles

Cortex - A

Highest performance

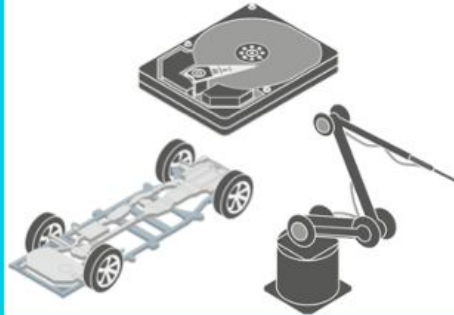
Optimised for
rich operating systems



Cortex - R

Fast response

Optimised for
high performance,
hard real-time applications



Cortex - M

Smallest/lowest power

Optimised for
discrete processing and
microcontrollers



ARM Assembly (in one slide)

Registers
r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
xpsr

Example Instructions:

- Arithmetic:

```
add r0, r1, r2
```

- Load:

```
ldr r0, [sp]
```

- Store

```
str r0, [sp, 4]
```

- Branch

```
b1 0x0800144
```

Explanations:

Add r2 to r1 and store result in r0

Load the value stored at sp into r0

Store the value of r0 into the location pointed to by sp+4

Store address of next (sequential) instruction into lr and jump to 0x08000144

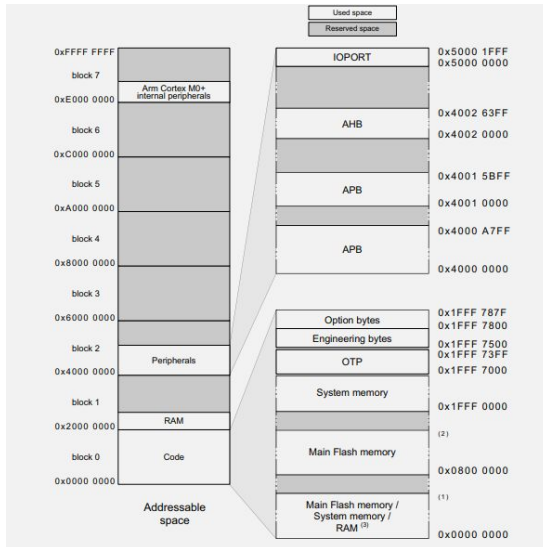
Stores return address of last call

Equals to RIP on X86_64

Current Program Status Register

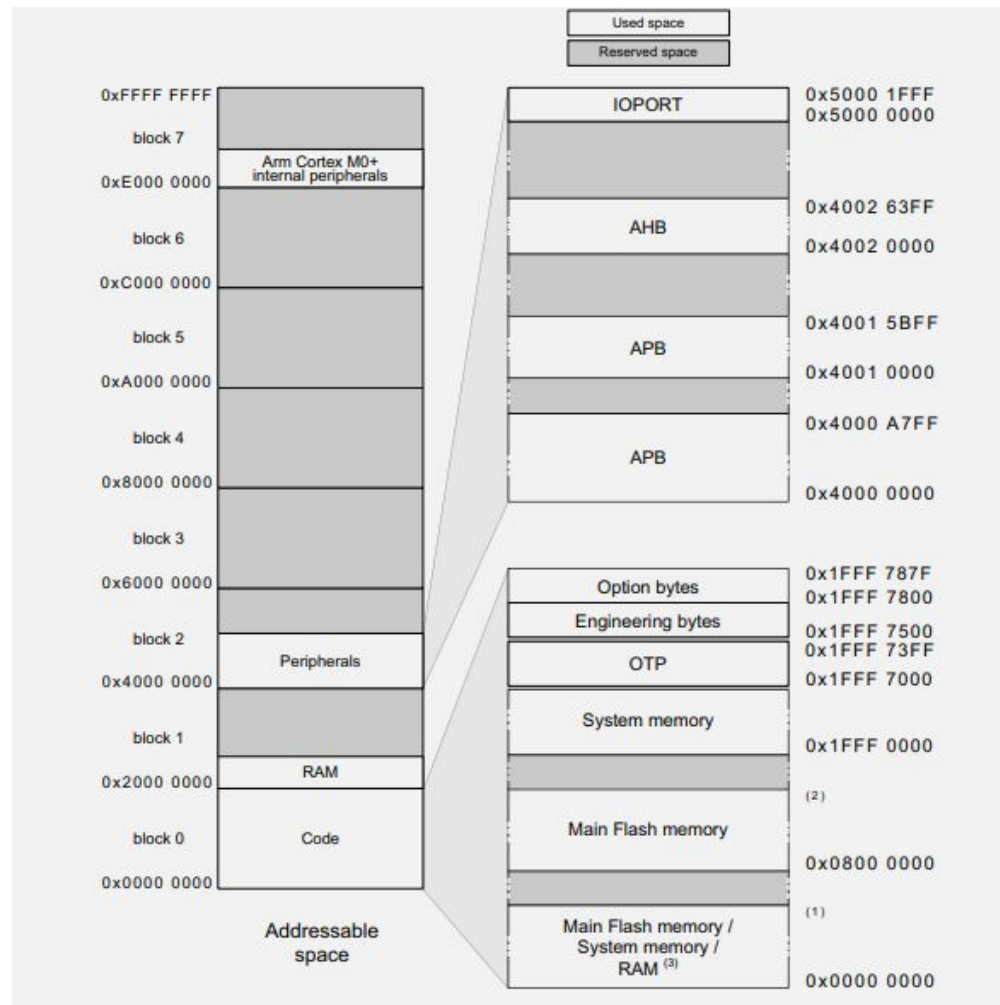
More in supplementary materials

Memory Maps



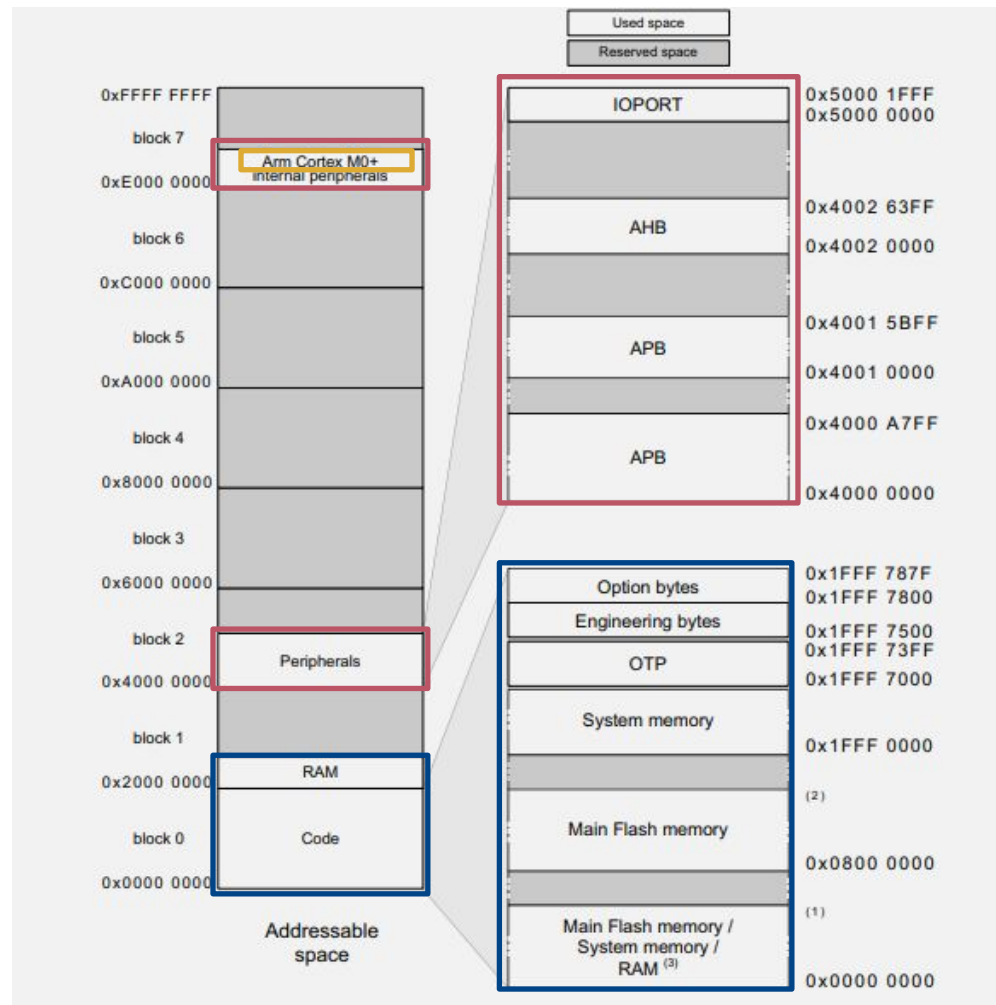
- Deeply embedded firmware has typically no need or resources for virtual address spaces
- Instead:
 - Single physical address space
 - Everything mapped at predefined addresses
- Memory Maps define the location of memory & peripherals

STM32G0x1 Memory Map



STM32G0x1 Memory Map

- Memory
- Processing Unit
- Peripherals



STM32G0x1 Memory Map

Firmware programming models:

Super-Loop Architecture

- A single loop for everything:
 - I/O, calculations
 - Returns never, or only on error case
- Simple logic
- Hard to scale:
 - Communication between subsystems
 - Interrupts
 - Starved inputs

```
void main() {  
    initialize_application();  
    while (1) {  
        collect_inputs();  
        do_calculations();  
        send_outputs();  
    }  
}
```

Firmware programming models: RTOS

- Different tasks for different subsystems
 - Logic is pushed into individual tasks
- Primitives for inter-task communication
 - E.g., Message Queues
- Scheduler can enforce real-time constraints
 - May use help of timer interrupts

```
void app_main() {  
    xTaskCreate(&hello_task, "hello_task", 2048, NULL, 5, NULL);  
    xTaskCreate(&blinky, "blinky", 512, NULL, 5, NULL);  
}
```

FreeRTOS Hello World Example

```
void hello_task(void *pvParameter) {  
    while(1) {  
        printf("Hello world!\n");  
        vTaskDelay(100 / portTICK_RATE_MS);  
    }  
}
```

Interrupts (IRQs)

- Events requiring attention from CPU
 - Generated by hardware or software
 - Processing may be disabled (masked)
- Typically associated with numbers
 - Optionally also with priorities
- Examples:
 - Arrival of new data on UART port
 - Expiry of timer
 - System calls from software



Interrupt Vector Tables

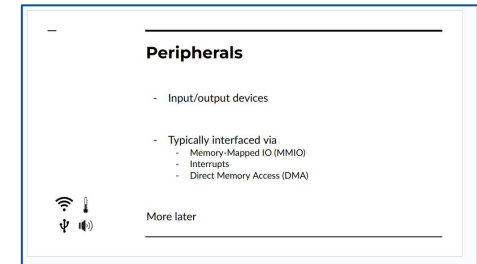
Offset	Purpose
0	SP on reset
4	Reset
8	NMI
12	HardFault
16	MemManageFault
20	BusFault
24	UsageFault
...	...

**First entries of
Cortex-M3 vector table**

- Determines how to handle incoming interrupts
- Maps interrupt numbers to Interrupt Service Routines (ISR)
- Different implementation on different architectures:
 - Address of ISR
 - Instruction for jumping to ISR
- Reprogrammable during run-time
 - E.g., by writing VTOR on ARM Cortex-M

Interacting with Peripherals

- Interrupts
 - Channel from peripheral to CPU
- Direct Memory Accesses (DMA)
 - Allows direct data transfer between memory & peripheral
 - CPU only needed to initiate DMA transfer
- Memory-Mapped I/O (MMIO)
 - Directly interfaced via memory
 - Organization in registers



Memory-Mapped I/O

Address	Type	Width	Reset value	Name	Description
UartBase + 0x00	Read/write	8	0x--	UARTDR	Data read or written from the interface.
UartBase + 0x04	Read/write	4/ 0	0x00	UARTRSR/ UARTECR	Receive status register (read)/ Error clear register (write).
UartBase + 0x08	Read/write	7	0x00	UARTLCR_H	Line control register, high byte.
UartBase + 0x0C	Read/write	8	0x00	UARTLCR_M	Line control register, middle byte.
UartBase + 0x10	Read/write	8	0x00	UARTLCR_L	Line control register, low byte.
UartBase + 0x14	Read/write	8	0x00	UARTCR	Control register.
UartBase + 0x18	Read	8	0x9-	UARTFR	Flag register (read only).
UartBase + 0x1C	Read/write	4/ 0	0x00	UARTIIR/ UARTICR	Interrupt identification register (read)/ Interrupt clear register (write).

ARM PrimeCELL UART (PL010) register summary (excerpt)

Register Types:

- Status
- Data
- Control

Status & Control registers
may be combined!

Memory-Mapped I/O

Address	Type	Width	Reset value	Name	Description
UartBase + 0x00	Read/write	8	0x--	UARTDR	Data read or written from the interface.
UartBase + 0x04	Read/write	4/ 0	0x00	UARTSR/ UARTECR	Receive status register (read)/ Error clear register (write).
UartBase + 0x08	Read/write	7	0x00	UARTLCR_H	Line control register, high byte.
UartBase + 0x0C	Read/write	8	0x00	UARTLCR_M	Line control register, middle byte.
UartBase + 0x10	Read/write	8	0x00	UARTLCR_L	Line control register, low byte.
UartBase + 0x14	Read/write	8	0x00	UARTCR	Control register.
UartBase + 0x18	Read	8	0x9-	UARTFR	Flag register (read only).
UartBase + 0x1C	Read/write	4/ 0	0x00	UARTIIR/ UARTICR	Interrupt identification register (read)/ Interrupt clear register (write).

ARM PrimeCELL UART (PL010) register summary (excerpt)

Register Types:

- Status
- Data
- Control

Status & Control registers
may be combined!

Typical Peripheral Accesses

- Configuration:
 - Firmware initialization
 - Before and after peripheral usage
- Status:
 - After configuration for confirmation
 - To check operability
- Data:
 - When data is to be sent or received

Problem: When should we read data?

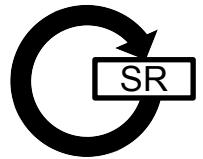
Peripheral Data Read: IRQ-based

- When data arrives:
 - Raise interrupt
 - Read and store data in ISR
- Pro:
 - Data arrival is ensured
- Con:
 - Asynchronous processing adds complexity
 - Potential to starve main firmware task



Peripheral Data Read: Polling-based

- When data is expected:
 - Loop over status register to check data availability
 - If data is available, read data register
- Pro:
 - No sudden changes of control-flow
 - Data is only received when desired
- Con:
 - Arrival of data can easily be missed
 - Lots of time spent in poll-loops



Example: UART Read

```
EXPORT serial_getc
serial_getc
PUSH    {R4,R5,R7,LR}
LDR     R5, [R0,#4]
MOV     R4, R0
```

```
loc_8006982
MOV     R0, R4
BL      serial_readable
CMP     R0, #0
BEQ     loc_8006982
```

```
LDR     R0, [R4,#0xC]
LDR     R2, =uart_handlers
MOVW    R1, #0x1FF
CMP     R0, #0
MOV.W   R0, R5, LSL#6
IT      EQ
MOVEQ   R1, #0xFF
LDR     R0, [R2,R0]
LDR     R0, [R0,#4]
ANDS    R0, R1
POP     {R4,R5,R7,PC}
; End of function serial_getc
```

```
EXPORT serial_readable
serial_readable

var_4= -4

SUB     SP, SP, #4
LDR     R0, [R0,#4]
LDR     R1, =uart_handlers
LSLS    R0, R0, #6
LDR     R0, [R1,R0]
LDR     R1, [R0]
LSLS    R1, R1, #0x1C
BPL     loc_8006CBA
```

```
LDR     R1, [R0]
STR     R1, [SP,#4+var_4]
LDR     R1, [R0,#4]
STR     R1, [SP,#4+var_4]
LDR     R1, [SP,#4+var_4]
```

```
loc_8006CBA
LDR     R0, [R0]
UBFX.W  R0, R0, #5, #1
ADD     SP, SP, #4
BX      LR
; End of function serial_readable
```

Overview

- Introduction to Firmware
 - Definition
 - Classification
- Inner workings
 - ARM ISA
 - Memory Maps
 - Programming Models
 - Interrupts
 - Hardware interactions
- Extraction
 - Firmware File Formats
 - Showcased Techniques
- Recap

Firmware File Formats

- Desktop systems have standardized file formats
 - E.g., EXE, ELF
- Firmware does not!
 - Vendors define and implement their own formats
 - Rarely documented
- Hard to distinguish code from data
 - Usually everything is interleaved
 - Large binary blobs



Firmware Unpacking

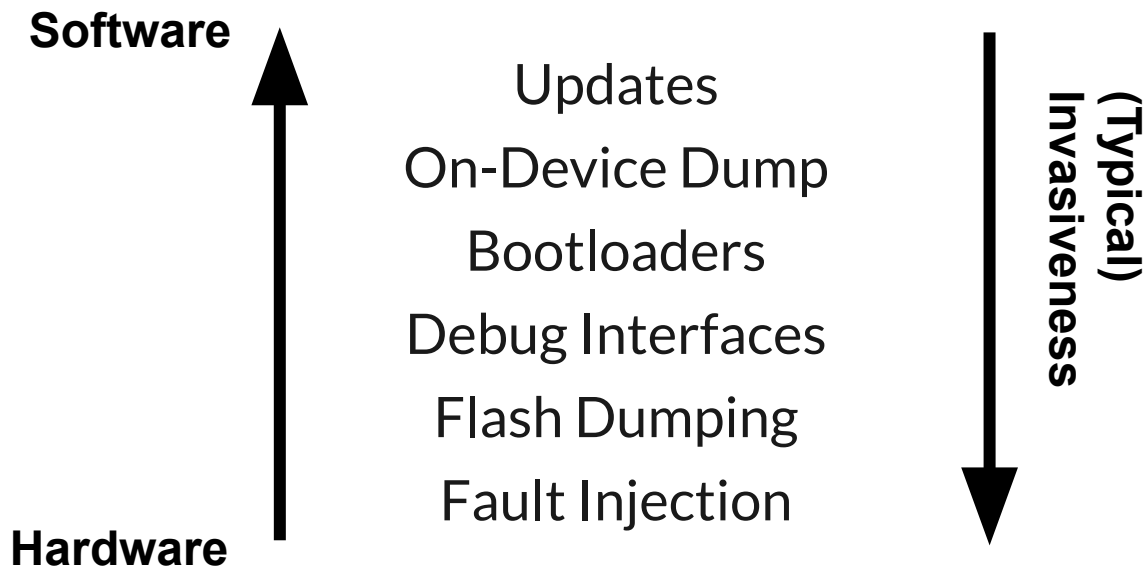
The process of restoring sections and, if applicable, files from a binary firmware image is called **unpacking**.

- Various signature and heuristic-based tools:
 - E.g., Binwalk
- Manual RE for unpacking may be required

Firmware Extraction

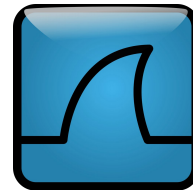
- Firmware is rarely open source
- Even obtaining the blobs may be difficult
 - Missing I/O interfaces
 - Read protections
 - Encrypted images
- Following is a showcase of techniques
 - Common techniques
 - Not conclusive
- If none of the methods apply, be creative!

Firmware Extraction Showcase



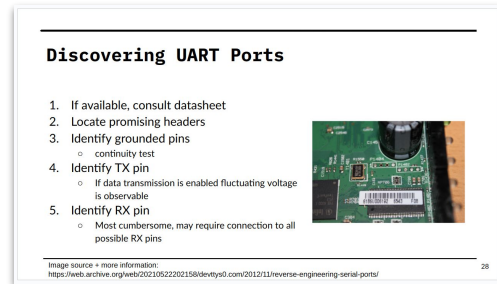
Firmware Updates

- Lots of embedded system foresee firmware updates
- Typical channels:
 - Network connections
 - SD cards/flash drives
 - USB connection
- Updates may be downloadable from vendor website
- Device initiated retrieval is sniffable!
 - Wireshark



On-Device Dump

- Complex devices (especially Type-I) may contain:
 - Flash storage accessible from software
 - Debug shell
- Likely required to find UART ports for shell access
 - Some systems may offer remote shell access
 - Or exploitation of application software ;)
- Once shell access is gained:
 - Dump using standard CLI utilities
 - E.g., dd or xmodem



Bootloaders

- Special case of on-device dump and/or updates
 - Booting into bootloader mode may require hardware access
- A bootloader is special code that:
 - May run before everything else on the system
 - May allow to re-flash firmware
 - May allow to read firmware

```
U-Boot 2018.09 (Oct 04 2018 - 05:36:06 -0700)

DRAM: 128 MiB
RPI Zero (0x900093)
MMC: sdhci@7e300000: 0
Loading Environment from FAT... *** Warning - bad CRC, using default environment

In: serial
Out: vidconsole
Err: vidconsole
Net: No ethernet found.
starting USB...
USB0: scanning bus 0 for devices... 2 USB Device(s) found
       scanning usb for storage devices... 0 Storage Device(s) found
Hit any key to stop autoboot: 0
U-Boot> █
```

Bootloaders: DFU

Many devices support a “Direct Firmware Upgrade” mode

- Standardized across vendors [[link](#)]
- Tooling available, e.g., `dfu-util`, `dfu-tool`

Usually Upgrade delivered via USB

- Device may be booted into a special dfu mode
- Sometimes: Bluetooth, 6loWPAN, ...

Not all vendors/implementations play by the standard

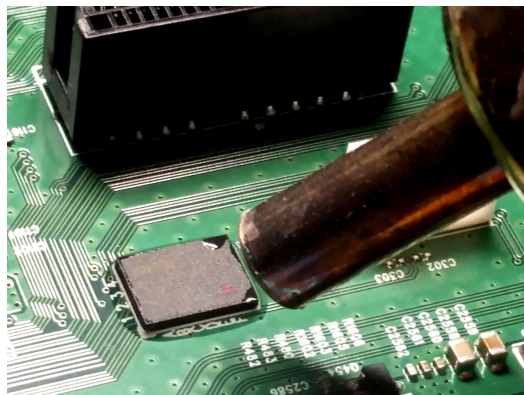
Debugging Interfaces

- Debugging interfaces can usually read memory
 - Trivial to dump firmware from here
- Wide-spread interface: JTAG
 - Variety of JTAG debuggers with vendor-specific software
 - Open source implementation: OpenOCD
- Hardware may provide read-out protections
 - Debug port enabled, but memory is unreadable
 - Can be bypassable in praxis, eg:
 - <https://blog.includesecurity.com/2015/11/NordicSemi-ARM-SoC-Firmware-dumping-technique.html>
 - <https://www.aisec.fraunhofer.de/en/FirmwareProtection.html>



Flash Dumping

- Depending on device-type, code may be on chips
 - E.g., SPI, nand flash
 - Sometimes encrypted
- Specialized tools for reading/writing exist
 - In easiest case: access via special clip
- Frequently requires soldering:
 - Wires to Pins of flash chip
 - Removal of chip from board
 -



Fault Injection (FI)

- Main idea: altering the intended behavior via faults
 - Also known as glitching
- Example Applications:
 - Skipping readout protection checks
 - Forcing outputs to be longer
- Example Techniques:
 - Voltage Fault Injection
 - Electromagnetic Fault Injection
- Likely requires removal of electronic components

More later!

Recap

- Introduction to Firmware
 - Definition
 - Classification
- Inner workings
 - ARM ISA
 - Memory Maps
 - Programming Models
 - Interrupts
 - Hardware interactions
- Extraction
 - Firmware File Formats
 - Showcased Techniques
- Recap

Recap

- Definition & Classification
 - "What" is firmware?
- ARM ISA, Memory Map, Programming Models, Interrupts & Peripheral Interaction
 - "How" does it work?
- Extraction Techniques
 - "Where" do we get it?